JackThomasNortheastern / cs5004-assignments

Code Issues Pull requests Actions Projects Security Insights Settings

main cs5004-assignments / instructions / README_A4.md

JackThomasNortheastern Create README_A4.md b803343 · 9 minutes ago

210 lines (144 loc) · 18.3 KB

Preview Code Blame Raw

# Domain Name Information - Instructions

The internet is made up of a series of protocols, one of the most notable protocols being the "Internet Protocol". This is the protocol that allows computers to communicate with each other over the internet. These are numbers often seen as 129.47.21.1 (as an example). However, the internet is not just a series of numbers, it is also a series of names. These names are called domain names. Domain names are human-readable names that are used to identify a website. For example, the domain name for Northeastern University is "northeastern.edu". To connect names to numbers, the internet uses a series of servers called Domain Name Servers (DNS). These servers are responsible for translating domain names into IP addresses, and combined with other servers that help with routing, allow you to connect to the website you want to visit.

For this assignment, we will explore keeping our own file of domain names and their associated IP addresses. This file will not only contain the domain name and IP address, but also the location of the server that sent the request. It is worth noting the web is a bit more complicated than this, as often a website like Northeastern, may have multiple servers throughout the world. This is done to help with speed and reliability. However, for our purposes, we will just assume one server and one location for our files.

## Learning Objectives

- Practicing Test Driven Development
- Implementing junit tests for all methods
- Writing out in different file formats (CSV, JSON, XML, TXT)
- "Serializing" and "Deserializing" data (converting data to a file format and back)
- Using third party libraries to assist in serialization/deserialization (Jackson and/or Open CSV)
- Exploring network connections in java and using them to connect to a server
- Encapsulating data between components, making use of immutable and mutable structures

where important

## Instructions

For this assignment, while we have provided example code we have written, including files you are welcome to use - you are free to fully design this project from the ground up as long as you:

- Use `DNInfoApp.java` as the main entry point of your program, and implement the argument options below.

When designing your application, you want to make sure much of what you provide is immutable or unmodifiable, except for in the actual containing class as you pass information around to different components of your program. This is a good practice in software engineering, and one of the learning objectives of this assignment.

The goal of the application is to provide Domain Name (URL) Information back to the client, based on the command line arguments provided. This information will be downloaded from the internet, and stored in a file. Once stored in a file, you do not have to update / download again (which is more common due to IP addresses shifting to balance load). As such, you are looking up the information in the file, downloading *and updating the file* if the domain name (hostname) is not found, and returning the information. The records/local file will be kept as an XML file.

Automatically updating a local file based on information obtained online is very common to prevent both unneeded web requests, and to make sure there are local copies. Most DNS servers and routing tables work in a similar way, though they are more complex and have mechanisms to continually update as locations change. To get information about the hostname, we are using a combination of built in java libraries and the ipapi.co API. This API is free to use up to a certain number of requests (1000 / day, 30,000 / month), so for use in this class you should be fine being in the free tier.

### Display Formats

For displaying the information to the client, you will have to display it based on different formats provided, namely, the data can be displayed in a pretty print format (provided), or in a CSV, JSON, or XML format. For all three formats, you should use the Jackson Library to serialize and deserialize the data. We have already included the updates to your build.gradle, so the libraries are included. If you are curious, they are in the dependencies section of the build.gradle file.

In addition to displaying the data to the stdout, you will also need to be able to write the data to a provided file.

> Note

> What does serialize / deserialize mean? It is a fancy way of saying "convert to a file format" and "convert from a file format". This is a common term in software engineering, and you will see it often. The file format can be a binary file, but in this case we are using human readable text file formats. Often it is used when the class fields are bound to specific data in the output, so they can be easily converted back and forth.

## Command Line Argument Options

The program should be able to take in the following command line arguments in any order.

- `-h` or `--help` - Display the help menu and end the program, we have provided this menu for you in the `ArgsController.java` file.
- `-f format` - The format to write out to. The options are `pretty`, `csv`, `json`, `xml`. If this is not provided, you should default to pretty.
- `-o file | stdout` - The output file to write to. If this is not provided, you should write to `stdout` (System.out).
- `--data` - This sets the 'database' file to read from. If it is not provided, it is `data/hostrecords.xml`.
- `hostname | all` - it assumes anything that doesn't have one of the above options is a hostname - just the first one that shows up. If `all` is typed in, it displays all contents of the file. If hostname is omitted it will display all contents of the file. The hostname is a part of the url, such as www.google.com or www.github.com or even google.com or github.com - though it is fine to treat www.google.com and google.com as separate entries in your file.

You should try these command line arguments with the sample application provided, so you can get a better understanding of what should be displayed.

> Tip
>
> We have included a sample completed program you can try out. No promise that it is 100% bullet proof, and probably some typos, but you can get the idea of how the final program works. If you go into the sample_working directory using your terminal, you can run the program with the following command:
>
> ```
> bin/DNInfo
>  or if windows
> bin\DNInfo.bat
> ```
>
> Both commands will execute the .jar file in the lib directory while properly setting the path. For macOS and linux, you may need to add the execute permission to the file ( `chmod +x bin/DNInfo` ).
>
> This program uses *extensive* command line arguments (in fact it is only a command line program). Use -h to see the help menu.

```
bin/DNInfo -h
```

## What we provided

We have provided a number of files for you. You DO NOT have to use any of the files except for `DNInfoApp` as your starting point. You do not have to design it the same way. We just left some of our packages there to help you get started/get hints. With that said, there are definitely some components you will want to work with, and we encourage you running/modifying the couple test files provided to get a better understanding of the jackson library, along with the network utilities we provided.

- `DNInfoApp.java` - The main entry point of your program. You need to keep this file, and start here for your program
- `controller/ArgsController.java` - A class to help you parse the command line arguments. We provided it, so you can have a premade help menu (saves copy and paste errors).
- `model/DomainNameModel.java` - an interface for the domain name model. We made use of internal classes such as DNRecord as an example on how to use jackson annotations, and to pass information around the program, and also a 'getInstance()` method to get the instance of the model. We left in the other features of the interface that we implemented, so you can see our interface *BUT* once again, you are free to design your own.
- `model/net/NetUtils.java` - Contains utilities for getting IP from hostname, and then getting the data from Ipapi.co API. You will probably want to use this file as is. To better understand this file, we have also included `test/TestNetUtils.java` which you can run to see how the file works, and edit to get a better idea of how NetUtils works.
- `formatters/Formats.java` - Enum for the different formats.
- `formatters/DomainXMLWrapper.java` - If you use Jackson for XML, this class (or similar) is needed to get it formatted corrected
- `formatters/DataFormatter.java` - A class we used to help us format the data based on the provided format. Assumes a collection of DNRecords. We provided the 'pretty print' option, so you didn't have to worry about spacing or spelling with the autograder. You do not have to use this file, but make sure you follow the example pretty print format provided. You will want to look at `test/TestJacksonSerialize.java` to see how to use the Jackson library to serialize and deserialize data.

> **Caution**
>
> These are not all the files we used! Don't be fooled thinking it is complete. We had files to help us especially in the model including the main model file, along with a java bean for loading data from online and more. Really think about design, and if you want to even use what we provided, but no matter what you will be adding files! This assignment is intentionally open for your design. We just wanted to give you a jump start on some of the tougher topics that involved working with other libraries and network connections.

> **Warning**
>
> The most common error we see is that you don't first check to see if the record already exists in your file. Please note, if the record already exists, you do not have to use NetUtils to grab the data! You only need NetUtils if the url given isn't already in the saved data. Due to IP addresses changing based on your location, the autograder only uses preexisting locations / fixed locations.

## 🔥 Task 1: Design

Before you start writing, it is important to think about design. You DO NOT have to be perfect in your design, so we will come back to this step a few times.

1. First, become a detective and read through the files provided. Take notes on what you are seeing. This is a common skill in software engineering, and you will need to do this often as you work with other people's code. Report.md has specific questions on the code that may help you.
2. Go to DesignDocument.md and fill out (ONLY) the initial design sections.

> **Tip**
>
> You are free to use mermaid or any other UML tools you want, just make sure if you are using another UML tool, you properly link the image in the markdown file.

## 🔥 Task 2: Implement by Test Driven Development

After your initial design, you should seek to follow the TDD process. This means you should write tests first, and then implement the code to pass those tests. Or better stated, you should write *ONE* test first, implement, and repeat until you have written all your tests.

1. Figure out a number of tests by brainstorming (done in design)
2. Write **one** test
3. Write **just enough** code to make that test pass
4. Refactor/update as you go along
5. Repeat steps 2-4 until you have all the tests passing/fully built program

Note: you often don't know all the tests as you write. As such, it is alright to continue to expand your list. This is where people get stuck on TDD. They think they have to know **all** the tests before they start. You don't. You just need to know the next test, and then at the end you double check you have covered all code paths and have full coverage.

> Caution
>
> Make sure to commit as you development. The bare minimum commits would be after every test, but you probably will have additional commits especially at the beginning.

### 💡 Implementation Tips

- Testing - you when it comes to testing the main, remember you can call it directly from your test. You can also use `System.setOut` to capture the output to test it.

  ```
  ByteArrayOutputStream outContent = new ByteArrayOutputStream();
  OutputStream originalOut = System.out; // just so you don't loose it
  System.setOut(new PrintStream(outContent));
  // Call your method that involve printing
  System.setOut(originalOut); // reset the output
  assertEquals("Your expected output", outContent.toString());
  ```

- It is also alright to remove spaces when testing/formatting. We do that same in our autograder, and while testing spaces exactly is important in some cases, it is not in this case.
- Make sure to run the tests we provide and play with them. This is an example of using tests to help you understand how third party libraries work before integrating them into your own code. They are never the tests we wrote to make sure we were using Jackson correctly.
- Jackson is a really powerful library that bases a lot of what it does on annotations and reflection. This means it seems to 'just work', if you set it up correctly. Here are some resources to help you get started:
  - https://github.com/FasterXML/jackson?tab=readme-ov-file
  - https://github.com/FasterXML/jackson-databind
  - https://github.com/FasterXML/jackson-dataformat-xml
  - https://stackify.com/java-xml-jackson/
  - https://github.com/FasterXML/jackson-dataformats-text/tree/master/csv
  - https://cowtowncoder.medium.com/writing-csv-with-jackson-204fdb3c9dac
  - https://stackify.com/java-xml-jackson/
  - https://www.baeldung.com/java-pojo-class (Just because Java Bean and POJO or "plain old java object" gets tossed around in the above links a lot)
- Make sure to make use of polymorphism. System.out is an OutputStream, and so is FileOutputStream. You can use this to your advantage to make sure you are to use the same functions for both.
- Yes, most of your controller is parsing args. Once they are parsed, the code to run the program

itself can be very small if you design it correctly.

- Starting with a Java Bean, and then converting to a record is useful when loading information from the internet API. However, most information can stay as a record, as we aren't modifying information once it is in the file and loaded.

> ⚠ Warning
>
> If you modify one of our files, you need to add tests for it. If you do not modify a provided file, you don't need to add tests. This can be done without modifying our provided code, but you are welcome to modify it if it fits your overall design better.

### 🔥 Task 3: Finish Design Document

By this point, your design has probably changed (very few people have perfect designs the first iteration). Update your design document with the final design in the "final design" section. We want to see the history of your first design to your final design. That is a good thing.

### 🔥 Task 4: Finish Report.md

Inside of Report.md you will need to answer a series of questions about your program, and about the learning objectives for the module in general. Fill it out.

> Important
>
> A primary purpose of this activity is to get you working through a process in addition to writing code. In software engineering the process you follow is often just as important as the code you write. This is because the process is what allows you to work with others, and to be able to maintain and update code over time. It may seem tedious right now, but it is a skill that will pay off in the long run.

## Submission

When you are done, you need to submit your code to gradescope. Go back to Canvas, and click through the link that takes you to the Gradescope assignment. When you submit, you should be able to upload directly from your computer, but I have left github repository uploading available as well should you choose.

## 📝 Grading Rubric

1. Learning (AG)
    - Code compiles without issue
    - Code passes all tests
2. Approaching (AG)
    - Passes the style check.

3. Meets (MG)
    - README.md is filled out (name, etc)
    - DesignDocument (INITIAL) sections are filled out
    - All methods are tested with JUnit tests
    - Method contain proper javadoc comments (not just javadoc notation but proper wording in the comment)
    - Report.md technical questions are questions answered correctly.
4. Exceeds (MG)
    - Code is DRY (Don't Repeat Yourself)
        - Including making use of helping/utility classes to reduce duplication.
    - Student uses proper inheritance without duplication
    - Makes use of inheritance for print/file streams.
    - Methods include tests for edge cases in addition to happy path
    - Design document (FINAL) sections are filled out
        - The notation needs to be correct, and the TAs will double check the final design matches the final implementation.
    - Report.md Deeper Thinking question filled out
        - Includes at least two references/citations

Legend:

- AG - Auto-graded
- MG - Manually graded

## Submission Reminder 🚨

For manually graded elements, we only guarantee time to submit for a regrade IF you submit by the DUE DATE. Submitting late may mean it isn't possible for the MG to be graded before the AVAILABLE BY DATE, removing any windows for you to resubmit in time. While it will be graded, it is always best to submit by the due date, so you have full opportunity to improve your grade.

If you need a reminder about the grading policy, please review the syllabus and the canvas page on 'formative/summative' grading. This class uses a unique grading system that will allow you to be flexible with due dates and multiple resubmissions (if you submit with time for TAs to give feedback), but we also ask that you continue to work on the assignment until you get a full grade.

> ⓘ Caution
>
> For this class, we give about a month for the **available until date**. This means you will only have a few manual resubmission attempts, and most everyone uses at least one! As such it is essential you submit on time!

## Autograder Limitation

Currently the autograder is limited in how it can test. As such, when it comes across an error it just stops. This means that if you have multiple errors in your code, you may only see the first one. We are working on improving this, but for now, you will need to fix the first error, and then rerun the tests to see the next error. Eventually, if every test passes, you will get the single point. It also may give you points for valid style, while errors exist in the code - so really assume the first 2 points are done together.